

REST API

How Applications Talk to Each Other Over HTTP

What is REST API?

REST = REpresentational **State Transfer**

REST is a style for building APIs on top of HTTP.

- A REST API lets one program (a client) ask another program (a server) for data or actions, using standard HTTP methods: **GET, POST, PUT, DELETE.**

In this lesson, we focus on the basic building blocks of a REST-style HTTP API — not every rule of the original REST design.

Key Terms

- **API** — a defined way for one program to talk to another
- **Endpoint** — a specific URI/URL the API responds to, e.g. `/students/3`
- **Resource** — the "thing" an endpoint represents, e.g. one student
- **JSON** — the text format most REST APIs use to send data:

```
{ "id": 3, "name": "Jane Kim" }
```

URL vs URI

- **URI** = a string that identifies a resource
- **URL** = a URI that also tells you where to access it

Example

- **URI:** `/students/3`
- **URL:** `https://api.example.com/students/3`

In REST APIs

We often casually say “URL”, but the more precise term for a resource identifier is **URI**.

One Complete Example

- **Goal:** Get student 3's information from the server.
- **Request:** A browser asks for `/students/3`.
- **Response:** The PHP server finds student 3 and returns JSON like `{ "id": 3, "name": "Jane Kim" }`.

This is the basic flow of a REST API: request → process → response.

Request (sent by the client):

```
GET /students/3  
Host: api.example.com
```

Response (sent by the server):

```
HTTP/1.1 200 OK  
Content-Type: application/json  
  
{ "id": 3, "name": "Jane Kim", "major": "Computer Science" }
```

| Library Analogy

The internet = a big **library**. Each item (a student record, a photo, a post) is a **resource** with an address (URL).

- **GET** → look up a book's information (read)
- **POST** → add a new book to the catalog (create)
- **PUT** → replace a book's catalog entry with new information (update)
- **DELETE** → remove a book's listing from the catalog (delete)

REST gives client and server a shared, predictable way to talk to each other.

Resource-Based URLs

<code>/students</code>	→ All students
<code>/students/3</code>	→ One specific student
<code>/students/3/grades</code>	→ That student's grades

The URI identifies **what** you are asking about. The HTTP method identifies **what you want to do** with it.

IMPORTANT

HTTP Methods

Method	Purpose	Example
GET	Retrieve a resource; makes no changes	GET /students/3
POST	Create a new resource, or trigger a process	POST /students
PUT	Replace a resource entirely with new data	PUT /students/3
DELETE	Remove a resource	DELETE /students/3

PUT is meant to replace the whole resource (**PATCH** is the method for changing only part of it). Our PHP project uses this simplified four-method version.

IMPORTANT

Responses: Status Code + JSON

A response is not just "data" or "a status" — it is a **status code**, some **headers**, and usually a **JSON body**, together.

Code	Meaning	When
200 OK	Success	GET or PUT succeeded
201 Created	New resource created	POST succeeded
204 No Content	Success, no body	DELETE succeeded
404 Not Found	Resource does not exist	GET /students/999

Stateless

Each request must carry everything the server needs to understand it — the server does not rely on remembering earlier requests.

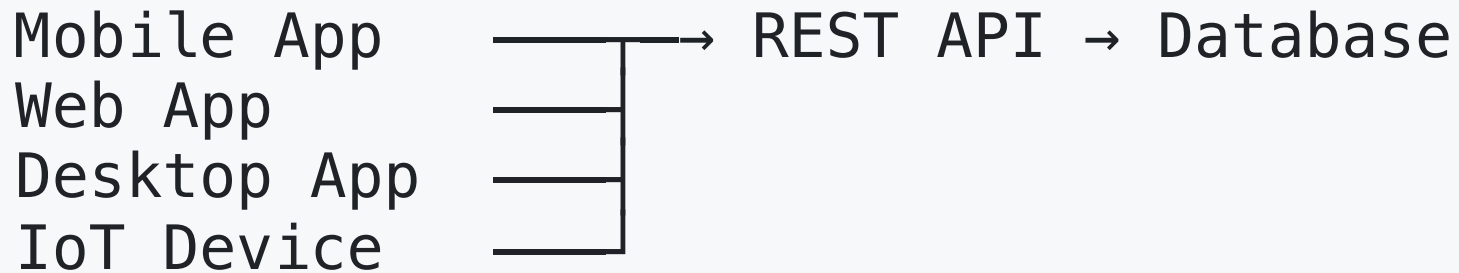
```
GET /students/3  
Authorization: Bearer token123
```

Stateless does **not** mean the server stores nothing.

- Student records still live permanently in a database — "stateless" only means the server does not need to remember the *conversation* between requests.

Why REST API Matters

1. Separation of Frontend and Backend



One API serves multiple applications!

2. Team Independence

- Frontend and backend teams can work and deploy independently
- The UI can change without changing the backend

3. Scalability

- Because each request is stateless, requests can be spread across many servers without any server needing to "remember" a specific client

4. Reusability

```
GET /students/3
```

- Web app: display a student profile
- Mobile app: show student info
- Admin panel: student management

REST is one of the most common API styles used across the industry — a useful skill for web development.

5. Technology Independence

- **Frontend:** React, Vue, Angular, Flutter, iOS, Android
- **Backend:** PHP, Node.js, Python, Java, .NET

Frontend and backend communicate through the REST API (HTTP + JSON). The backend then talks to its own database separately — for example, PHP using PDO.

- **Database:** MySQL, PostgreSQL, MongoDB

A Real-World Example: GitHub API

```
GET https://api.github.com/repos/microsoft/vscode
```

```
{ "full_name": "microsoft/vscode", "stargazers_count": 12345, "language": "TypeScript" }
```

This is a simplified request for illustration — the real API also requires things like authentication and pagination for many endpoints.

- Creating a new repository works the same way, using **POST** with a JSON body.

Coming Next

We will build this exact API — the `/students` endpoints — using PHP.

- Same resource, same HTTP methods
- You will see how PHP reads the request, looks up the data, and returns JSON